Code of the Day – Find the Non-repeating Element in an Array

- We are given an array in which all the numbers except one are repeated once. In other words, there are 2N + 1 numbers in the array where N numbers occur twice in the array. We need to find that number that occurs only once in the array.

- Example #1
  - Input = {1, 1, 3, 2, 4, 2, 3}
  - Output = 4

- Example #2
  - Input = {7, 6, 4, 3, 2, 4, 7, 2, 3}
  - Output = 6

```
#include <iostream>
int main() {
  int arr[] = {1,1,3,2,4,2,3};
  int size = sizeof(arr)/sizeof(int);
  int result = 0;
  for(int i =0 ; i < size; i++) {
    result = result ^ arr[i];
  }
  std::cout << "Number occurs once: " << result;
  return 0;
}
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

OpenMP – Synchronization and Reduction

02/28/2025

Before we get started…

- Quick overview, things discussed last time
  - Work sharing - sections and tasks
  - Scope of a variable
- Purpose of today's lecture:
  - Scope of a variable
  - Synchronization
- Miscellaneous
  - Assignment #3 is released – due on **Monday 3/3 at 23:50 PM**
  - Next Monday 3/3 TA will come to discuss assignment #4 and final project details (proposal due on 3/24)

Euler Cluster Policy – Extremely Important

- Visual Studio Code (in addition to variants such as Code-OSS, VSCodium, Cursor) can cause serious stability problems when running on networked filesystems
  - These can lead to system crashes and data corruption, so the use of such tools is strictly forbidden on Euler
- You should only use Euler as a place to compile and run your code (through Slurm Sbatch), instead of using VScode to program your code directly
- We have a few students banned from accessing Euler due to the misuse. Please make sure you follow the policy, or you won't be able to complete the assignment on time

Work Plan, OpenMP

Parallel Regions

Work sharing – Parallel tasks

Variable Scoping Issues

Synchronization

Performance Issues

The `private` Clause

- Variable[s] declared **`private`** in the `pragma` are reproduced for each thread
  - Consequence: each thread will have a private copy of that variable in the associated parallel region
  - NOTE: Private variables are un-initialized (typically may be initialized to zero in C++)

```
void work(float* c, int N) {
    float x, y;
    int i;
    #pragma omp parallel for private(x,y)
    for(i=0; i<N; i++) {
        x = a[i]; y = b[i];
        c[i] = x + y;
    }
}
```

  - The variables x and y are private to each thread – initial values are undetermined

`private` vs. `firstprivate`

- **`firstprivate`**
  - In addition to keep a private copy per thread, the copied variable is initialized using the value of the target

Example [dwelling on the `private` concept]

```
#include <omp.h>
#include <cstdio>
using std::printf;

int main() {
  int i = 10;

  #pragma omp parallel num_threads(4)
  {
    int me = omp_get_thread_num();
    printf("threadID = %d  i = %d\n", me, i);
  }

  return 0;
}
```

```
curie ~/ME759/OpenMP> g++ example.cpp  -fopenmp
curie ~/ME759/OpenMP> ./a.out
threadID = 0  i = 10
threadID = 2  i = 10
threadID = 1  i = 10
threadID = 3  i = 10
```

Example [dwelling on the `private` concept]

```
#include <omp.h>
#include <cstdio>
using std::printf;

int main() {
 int i = 10;

 #pragma omp parallel num_threads(4) private(i)
 {
   int me = omp_get_thread_num();
   printf("threadID = %d  i = %d\n", me, i);
 }

 return 0;
}
```

- Indeed, there is no guarantee you will always see i = 0. Without initialization, the value may just be garbage.

```
curie ~/ME759/OpenMP> g++ example.cpp  -fopenmp
curie ~/ME759/OpenMP> ./a.out
threadID = 0  i = 0
threadID = 2  i = 13456
threadID = 1  i = -1234
threadID = 3  i = 459371
```

Example [dwelling on the private concept]

```
#include <omp.h>
#include <cstdio>
using std::printf;

int main() {
 int i = 10;

 #pragma omp parallel num_threads(4) firstprivate(i)
 {
   int me = omp_get_thread_num();
   printf("threadID = %d  i = %d\n", me, i);
 }

 return 0;
}
```

```
curie ~/ME759/OpenMP> g++ example.cpp  -fopenmp
curie ~/ME759/OpenMP> ./a.out
threadID = 0  i = 10
threadID = 2  i = 10
threadID = 1  i = 10
threadID = 3  i = 10
```

The `lastprivate` OpenMP directive

- Ensures that the last iteration's value of a variable in a parallel loop or sections construct is copied back to the original variable after the parallel region ends

```
#include <cstdio>
#include <omp.h>

int main()
{
  int i = 10;
  #pragma omp parallel for num_threads(4) schedule(static, 2) lastprivate(i)
  for (int j = 0; j < 11; j++)
  {
    i = omp_get_thread_num();
  }
  printf("i = %d\n", i);
  return 0;
}
```

```
$ g++ test.cpp -Wall -fopenmp
$ ./a.out
i = 1
```

- What do we expect the master thread to print at the end?

Thread 0: Iterations 0, 1
Thread 1: Iterations 2, 3
Thread 2: Iterations 4, 5
Thread 3: Iterations 6, 7
Thread 0: Iterations 8, 9
Thread 1: Iteration 10
Thread that executes the last iteration: Thread 1.
Therefore, i=1.

Quiz: what is the output of this piece of code, and why?

```
#include <cstdio>
#include <omp.h>

int main()
{
  int i = 10;
  #pragma omp parallel for num_threads(4) schedule(static, 3) lastprivate(i)
  for (int j = 0; j < 11; j++)
  {
    i = omp_get_thread_num();
  }
  printf("i = %d\n", i);
  return 0;
}
```

Thread 0: 0, 1, 2
Thread 1: 3, 4, 5
Thread 2: 6, 7, 8
Thread 3: 9, 10

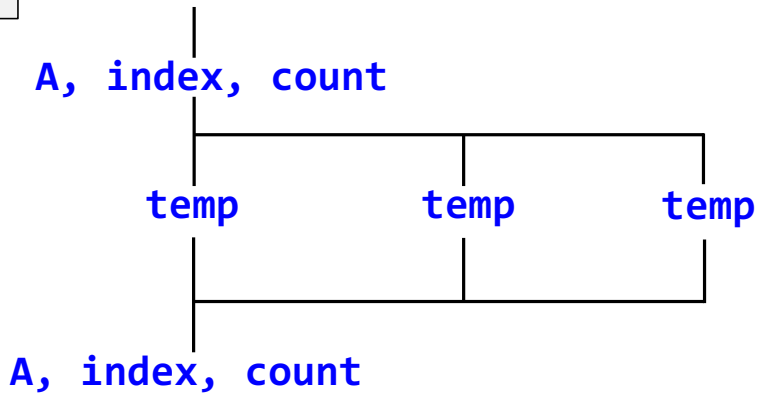
Quiz: what's shared and what's not?

```
float A[10];
main() {
  int index[10];
  #pragma omp parallel num_threads(3)
  {
    Work(index);
  }
  printf("%d\n", index[1]);
}
```

Assumed to be in another translation unit

```
extern float A[10];
void Work(int* index)
{
   float temp[10];
   static int count;
   <...>
}
```

A, index, and count are shared by all threads, but temp is private to each thread



Variable Scoping is a Common Source of Errors in OpenMP

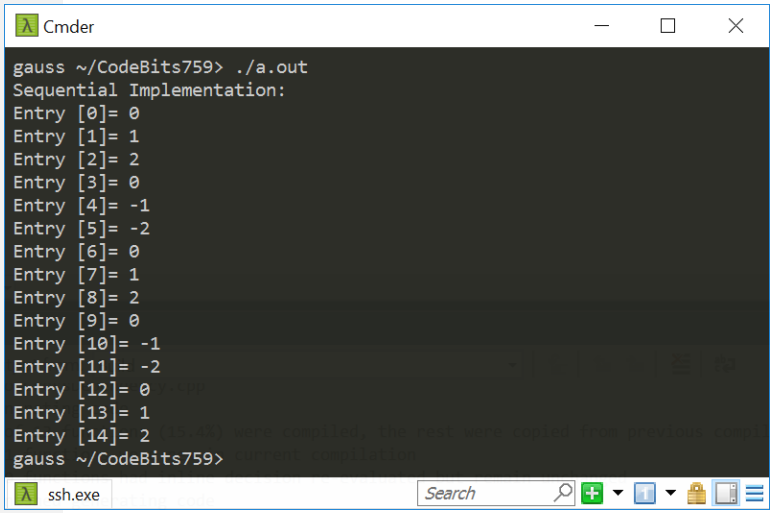
- When in doubt, you should
  - Explicitly indicate the scope of a variable: `shared`, `private`
  - Write a toy example to experiment and justify your thinking

- When parallel threads execute a parallel region, it is your responsibility to ensure:
  - Data dependencies are inexistent across parallel regions, or
  - If data dependencies exist, there will be no race conditions

- Compiler has limited knowledge about your parallelism and cannot identify data race
  - Again, it is your responsibility to avoid any misuse of data, dependencies, race condition, etc.
  - Though, many good tools exist to help you deal with this situation – thread sanitizer, leak detector, etc.

Data Dependency Example [1/5]

```
#include <omp.h>
#include <iostream>
#include <vector>

int main()
{
    const int N = 15;
    std::vector<int> a(N, 0); // create a vector of 15 zeros

    // initialize the first "offset" entries of a and then
    // populate all the other entries of a based on these first entries
    const int offset = 3;
    for (int i = 0; i < offset; i++)
        a[i] = i;

    for (int i = offset; i < a.size(); i++)
        a[i] -= a[i - offset];


    //print out the content of a
    std::cout << "Sequential Implementation:\n";
    for (int i = 0; i < a.size(); i++)
        std::cout << "Entry [" << i << "]= " << a[i] << std::endl;

    return 0;
}
```



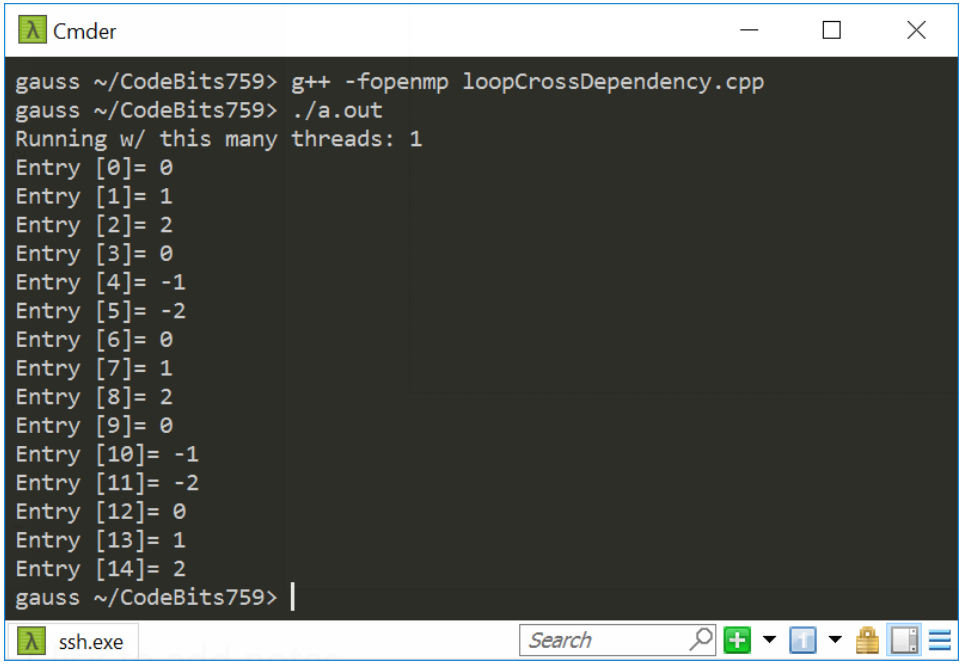
Data Dependency Example [2/5]

```
#include <omp.h>
#include <iostream>
#include <vector>

int main()
{
    const int N = 15;
    std::vector<int> a(N);

    //Basic idea: initialize the first "offset" entries of a and then
    //populate all the other entries of a based on these first entries
    const int offset = 3;
    for (int i = 0; i < offset; i++)
        a[i] = i;

    #pragma omp parallel num_threads(2)
    {
        #pragma omp single
        std::cout << "Running w/ this many threads: " << omp_get_num_threads() << std::endl;

        #pragma omp for
        for (int i = offset; i < N; i++)
            a[i] -= a[i - offset];
    }

    //print out the content of a
    for (int i = 0; i < a.size(); i++)
        std::cout << "Entry [" << i << "]= " << a[i] << std::endl;

    return 0;
}
```

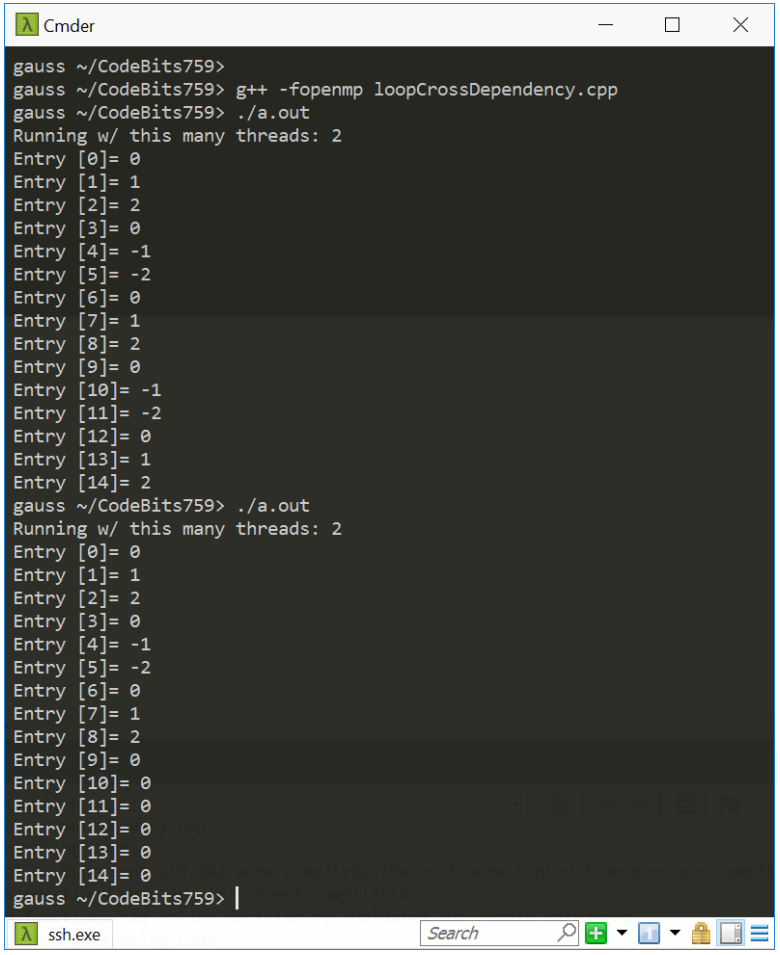
We're going to keep changing this value here

Data Dependency Example: 1 Thread [3/5]

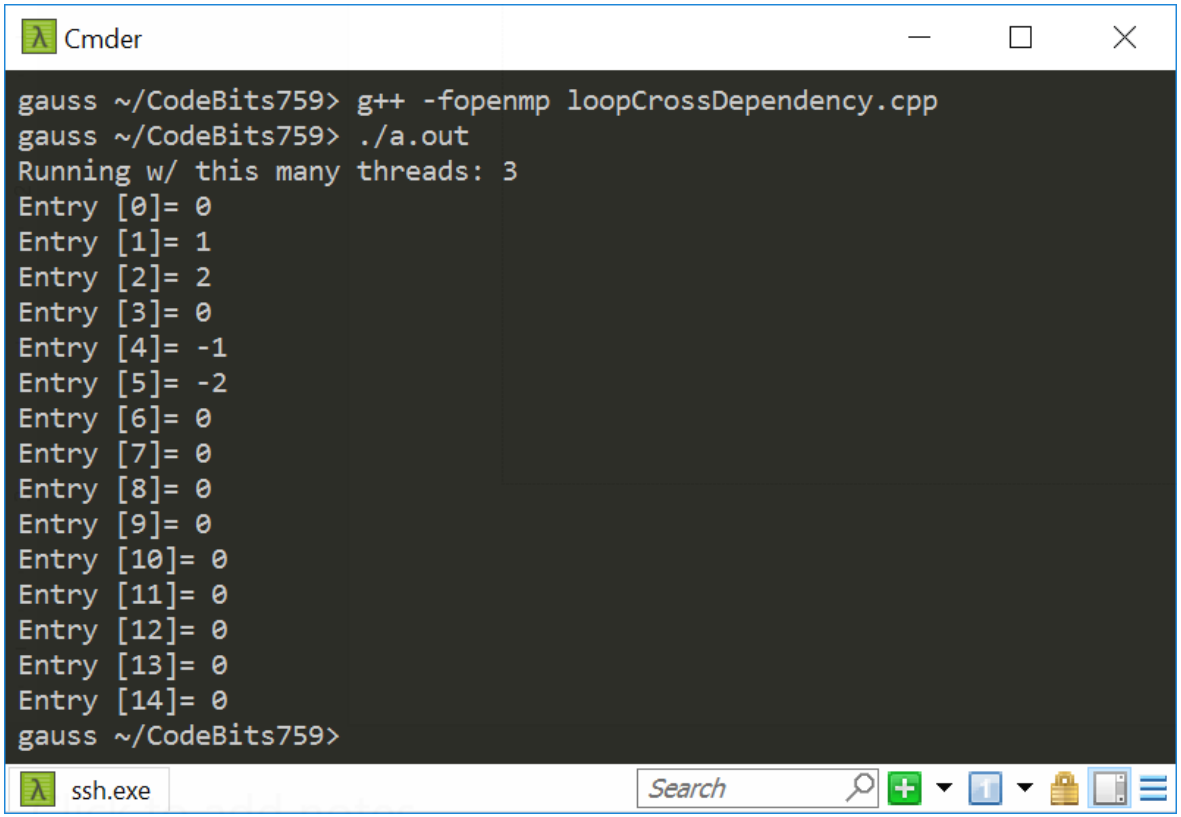
- Running OpenMP with one thread is essentially running sequentially



Data Dependency Example: 2 Threads [4/5]



Data Dependency Example: 3 Threads [5/5]



Work Plan, OpenMP

Parallel Regions

Work sharing – Parallel tasks

Variable Scoping Issues

Synchronization

Performance Issues

The `barrier` Construct

- The **`barrier`** construct places a synchronization point for all threads in the region
  - Each thread in the associated parallel region waits until all others to arrive at the barrier before each of them can move on
  - Can be used to ensure data or functional dependencies among tasks
- Example below assumes that in `DoSomework(X,Y)` , X is an input and Y is the output:

```
#pragma omp parallel shared(A, B, C)
{
    DoSomeWork(A,B); // input is A, output is B

    #pragma omp barrier

    DoSomeWork(B,C); // input is B, output is C
}
```

Explicit vs Implicit Synchronization

- What is the difference between the two code?
  - Note that the structured block of a parallel region places an implicit barrier where threads join

```
#pragma omp parallel shared(A, B, C)
{
    DoSomeWork(A,B);

    #pragma omp barrier

    DoSomeWork(B,C);
}
```

```
#pragma omp parallel shared(A, B, C)
{
    DoSomeWork(A,B);
}

#pragma omp parallel shared(A, B, C)
{
    DoSomeWork(B,C);
}
```

- The two code are functionally equivalent, i.e., they all produce the same, correct result
- However, barrier is typically implemented with spin lock which is much faster than joining threads

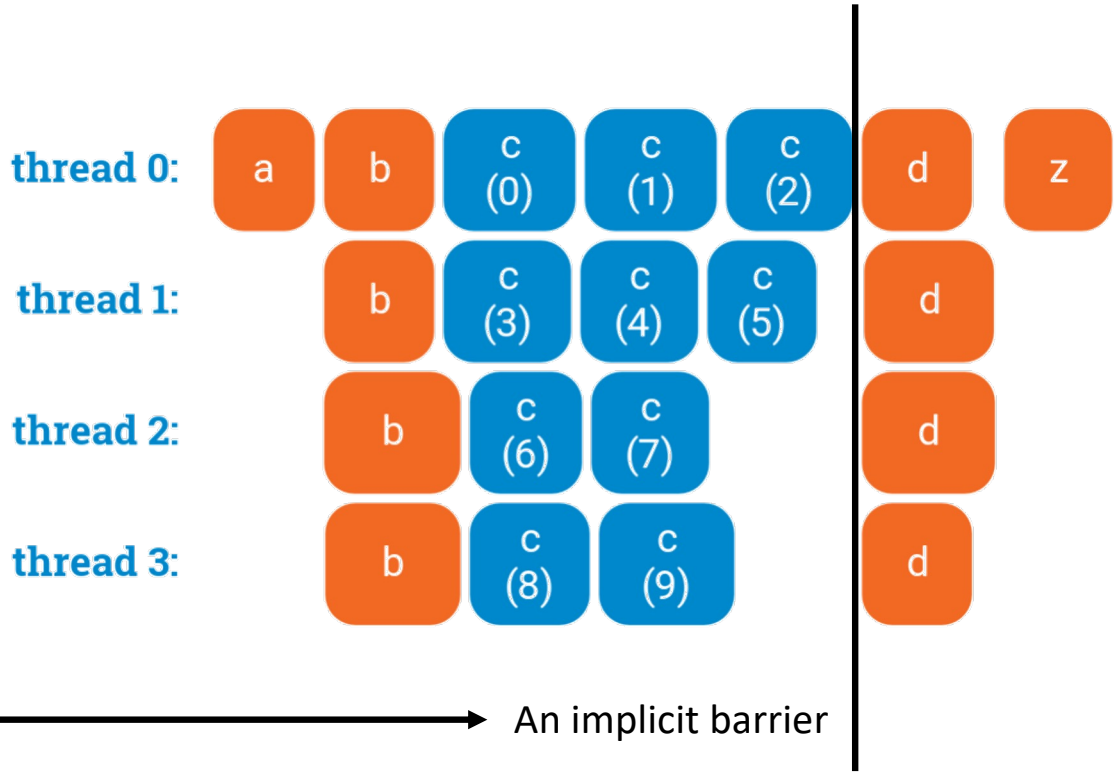
Implicit Barriers at the End of a Structured Block

- Many OpenMP constructs have *implicit* barriers when hitting the end of a structured block "}"
  - `parallel` – necessary barrier – cannot be removed
  - `for`
  - `single`
  - `sections`

- Unnecessary barriers may hurt performance and can be removed with the `nowait` clause
  - When a thread finishes its work and hit the end of a structured block, it doesn't need to wait for other threads to arrive but can move forward to the next task
  - The `nowait` clause is applicable to:
    - `for` directive
    - `single` directive
    - `sections` directive

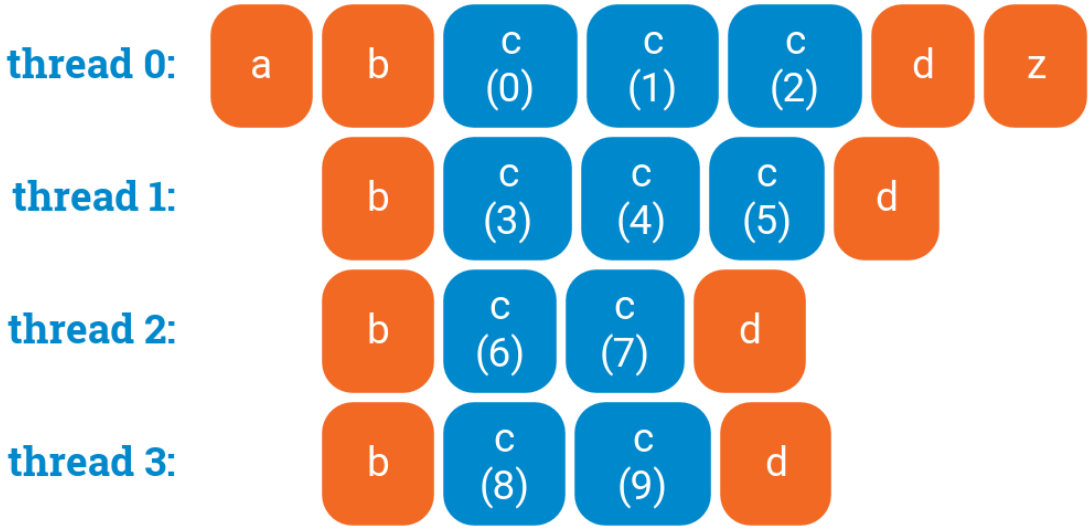
OpenMP Parallel For Loops: Waiting (default)

```
a();
#pragma omp parallel {
  b();
  #pragma omp for
  for (int i = 0; i < 10; ++i) {
    c(i);
  }
  d();
}
z();
```



OpenMP Parallel For Loops: No Waiting (with `nowait`)

```
a();
#pragma omp parallel {
  b();
  #pragma omp for nowait
  for (int i = 0; i < 10; ++i) {
    c(i);
  }
  d();
}
z();
```



Better performance due to improved asynchrony through `nowait`

The nowait Clause

```
#pragma omp for nowait
  for(…)
    { […] }
```

```
#pragma single nowait
  { […] }
```

- Use nowait when you do not need all threads to wait or synchronize at a point
  - Example: Some threads might work on the second for loop while some threads are still tied up with the first for loop

```
#pragma omp for schedule(static) nowait
 for(int i=0; i<n; i++)
   a[i] = bigFunc1(i);

#pragma omp for schedule(dynamic,1)
 for(int j=0; j<m; j++)
   b[j] = bigFunc2(j);
```

[IOMPP]→

Solving Data Race with Synchronization

- Race condition: two or more threads access a shared variable at the same time
  - Leads to nondeterministic behavior

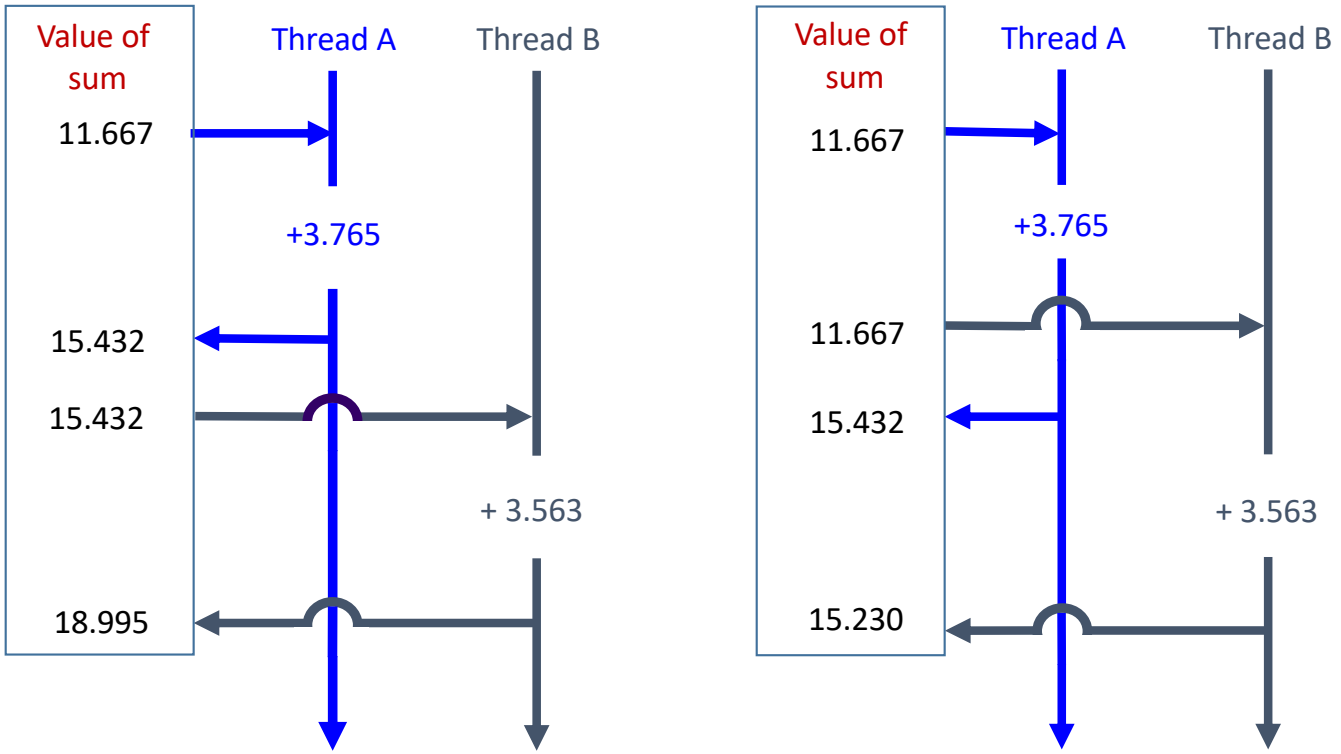
```
float dot_prod(float* a, float* b, int N)
{
  float sum = 0.0;
  #pragma omp parallel for
  for(int i=0; i<N; i++) {
    sum += a[i] * b[i];
  }
  return sum;
}
```

- Both Thread A and Thread B are executing the statement

```
sum += a[i] * b[i];
```

[IOMPP]→

Race conditions (data hazards) – two possible scenarios



Order of thread execution causes non-deterministic behavior in a data race

[IOMPP]→

Solving Data Race using the OpenMP `critical` Construct

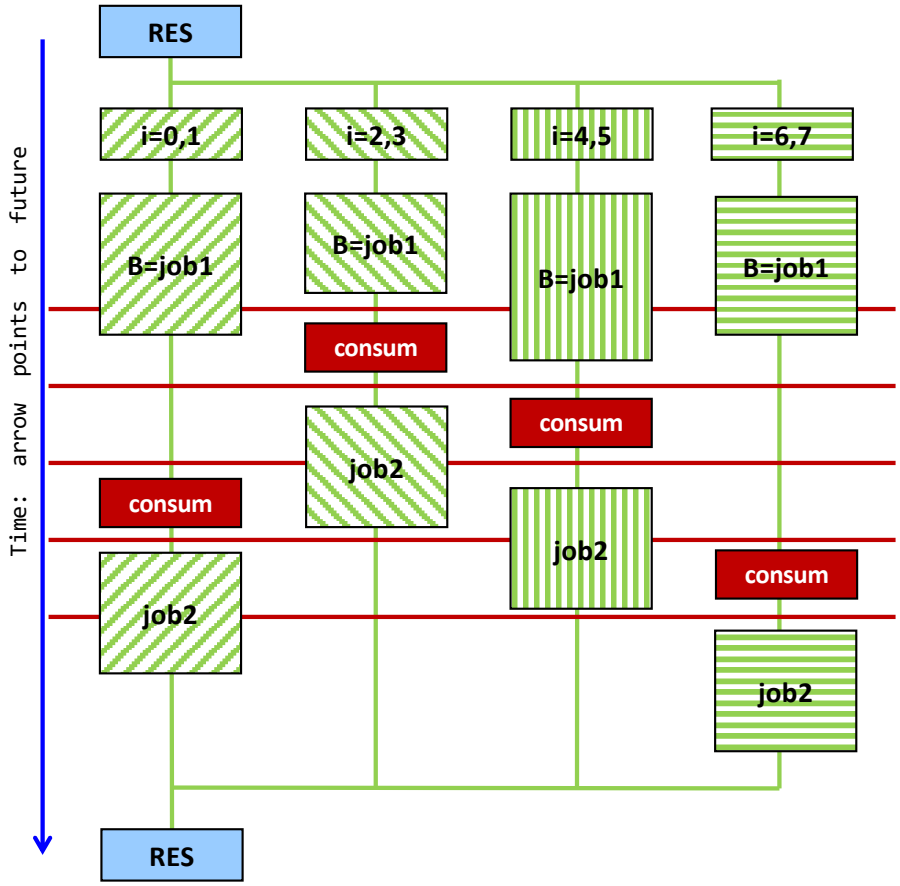
`#pragma omp critical[(name)]`: defines a critical region on a structured block

- The `critical` construct allows only one thread to enter its structured block at a time
  - Enable exclusive access
- Naming the critical construct `LOCK_SUM` is optional but a good idea for improved readability

```
float dot_prod(float* a, float* b, int N)
{
    float sum = 0.0;
    #pragma omp parallel for
    for(int i=0; i<N; i++) {
        #pragma omp critical(LOCK_SUM)
        sum += a[i] * b[i];
    }
    return sum;
}
```

Execution Diagram of an OpenMP `critical` Region

```
float* RES;

#pragma omp parallel
{
  #pragma omp for num_threads(4)
  for(int i=0; i<8; i++) {
    float B = job1(i);

    #pragma omp critical(RES_Lock)
    consum(B, RES);

    job2(B);
  }
}
```



Difference between OpenMP `atomic` and `critical`

- The `atomic` construct can be viewed as a special case of a `critical` section
  - However, atomic applies to simple update of memory (e.g., ++, --, &, |)
  - Atomic is less costly than `critical` as it typically has direct hardware ISA support

```
float dot_prod(float* a, float* b, int N)
{
    float sum = 0.0;
    #pragma omp parallel for
    for(int i=0; i<N; i++) {
        #pragma omp critical(LOCK_SUM)
        sum += a[i] * b[i];
    }
    return sum;
}
```

```
float dot_prod(float* a, float* b, int N)
{
    float sum = 0.0;
    #pragma omp parallel for
    for(int i=0; i<N; i++) {
        #pragma omp atomic
        sum += a[i] * b[i];
    }
    return sum;
}
```

Faster due to direct hardware support

Memory Operations Supported by OpenMP `atomic`

- The memory updates that can be protected by `atomic` must be one of:

```
x++;
++x;
x--;
--x;
```

```
x <op>= <expression>;
```

`<op>` can be one of: `+, *, -, /, &, ^, |, <<, >>`

- `<expression>` must not reference `x`
- Only the load and store of `x` is protected (not `<expression>`)

`atomic` and `critical`: Under the Hood

- "**Critical**" is typically implemented through *lock* (e.g., std::mutex)
  - Lock is computationally expensive in most cases, especially when you don't grab the lock and you need to wait (i.e., pre-empted by the operating system) until the lock is released
- "**`Atomic`**" is typically implemented with specialization instructions
  - A sequences of instructions that guarantee *atomic* accesses and updates of shared single word variables without the need of explicit lock (e.g., std::atomic in C++ template)

```
int sum(0);
std::mutex mutex;
#omp parallel num_threads(4)
{
  mutex.lock()
  sum += a[i] * b[i];
  mutex.unlock();
}
```

```
std::atomic<int> sum(0);
#omp parallel num_threads(4)
{
  sum.fetch_add(a[i] * b[i]);
}
```

Lock is typically much slower than atomic operations

The OpenMP `reduction` Construct

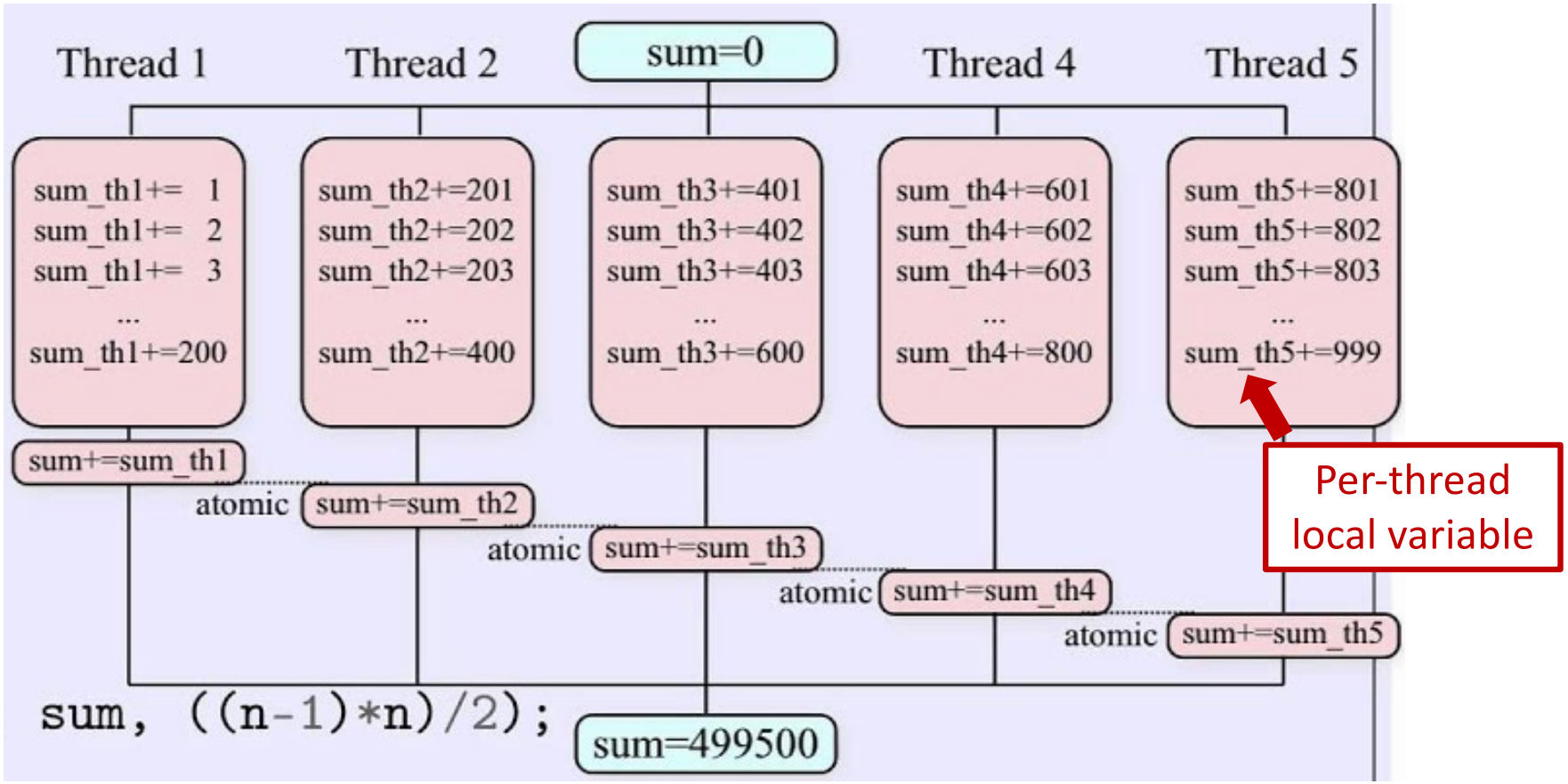
- Reduction is a type of operator that is commonly used in parallel programming to reduce the elements of an array into a single result
  - Ex: compute the sum of elements in a vector: [1, 2, 5, 0, -3] → 1+2+5+0+(-3) = 5

- OpenMP provides a `reduction` construct to perform parallel reduction operation

```
#pragma omp for reduction(op:list)
```

- Here's what happens inside the parallel reduction construct:
  - A private copy of each variable in the "list" is created and initialized depending on the "op"
  - These copies are updated locally by threads
  - At the end, local copies are combined through "op" into a single value stored back to the original list

Example: The OpenMP `reduction` Clause

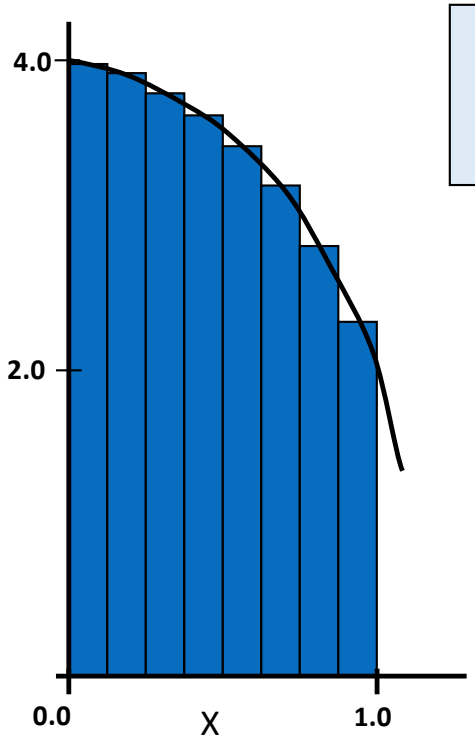


[IOMPP]→

Supported Operands by OpenMP Reduction

Operand	Initial value	Notes
+	0	Summation
*	1	Multiplication
-	0	Subtraction
&	~0	Bitwise AND
|	0	Bitwise OR
^	0	Bitwise XOR
&&	1	Logical AND
||	0	Logical OR
min/max	User-set	min/max of two variables

reduction Example: Numerical Integration



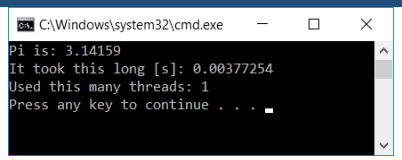
$$\int_0^1 \frac{4}{1+x^2}\,dx = \pi$$

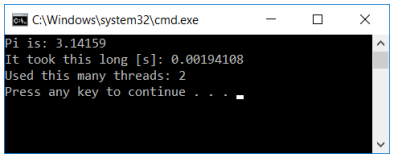
```
static long num_steps=100000;
double step, pi;

void main() {
   int i;
   double x, sum = 0.0;

   step = 1.0/num_steps;
   for (i=0; i< num_steps; i++) {
      x = (i+0.5)*step;
      sum = sum + 4.0/(1.0 + x*x);
   }
   pi = step * sum;
   printf("Pi = %f\n",pi);
}
```

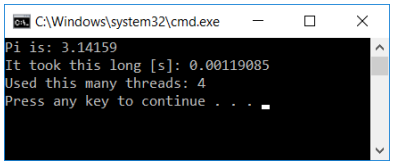
[IOMPP]→

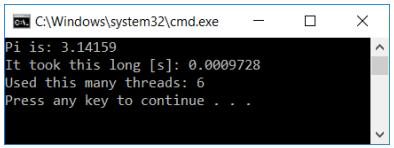
Scaling Analysis: Computation of π

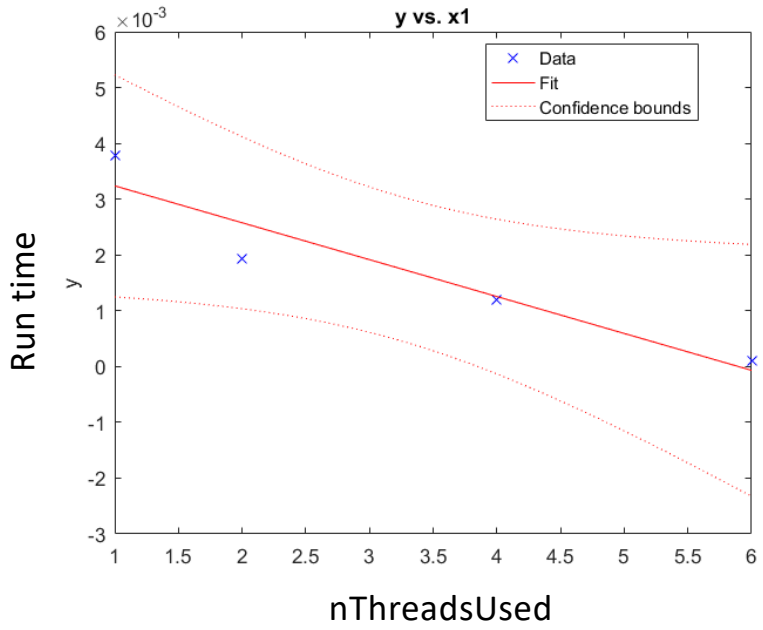
```
#include <omp.h>
#include <cstdio>

void main() {
   static long num_steps = 1000000;
   const double step = 1.0 / (double)num_steps;
   double sum = 0.0;
   double startT = omp_get_wtime();
   const int nThreadsUsed = 1;
   #pragma omp parallel num_threads(nThreadsUsed)
   {
       #pragma omp for reduction(+:sum)
       for (int i = 0; i < num_steps; i++) {
           double x = (i + 0.5)*step;
           double dummy = 1. / (1. + x*x);
           sum += dummy;
       }
   }
   double myPi = 4.*step * sum;
   double timeElapsed = omp_get_wtime() - startT;

   std::printf("Pi is: %f\n", myPi);
   std::printf("It took this long [s]: %f\n", timeElapsed);
   std::printf("Used this many threads: %d\n", nThreadsUsed);
}
```











Wait, it can get even better: the simd directive

```
#include <omp.h>
#include <iostream>
#include <chrono>
int main() {
    int num_steps = 1000000;
    const double step = 1.0 / (double)num_steps;
    double sum = 0.0;
    const int nThreadsUsed = 4;
    auto start = std::chrono::high_resolution_clock::now();
    #pragma omp parallel num_threads(nThreadsUsed)
    {
        #pragma omp for simd reduction(+:sum)
        for (int i = 0; i < num_steps; i++) {
            double x = (i + 0.5) * step;
            double dummy = 1. / (1. + x * x);
            sum += dummy;
        }
    }
    double myPi = 4. * step * sum;
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;
    std::cout << "Pi is: " << myPi << std::endl;
    std::cout << "It took this long [s]: " <<  elapsed.count() << std::endl;
    std::cout << "Used this many threads: " << nThreadsUsed << std::endl;
    return 0;
}
```

```
$ g++ test.cpp -O3 –fopenmp
$ ./a.out
Pi is: 3.14159
It took this long [s]:
0.000744621
Used this many threads: 4
```

No simd

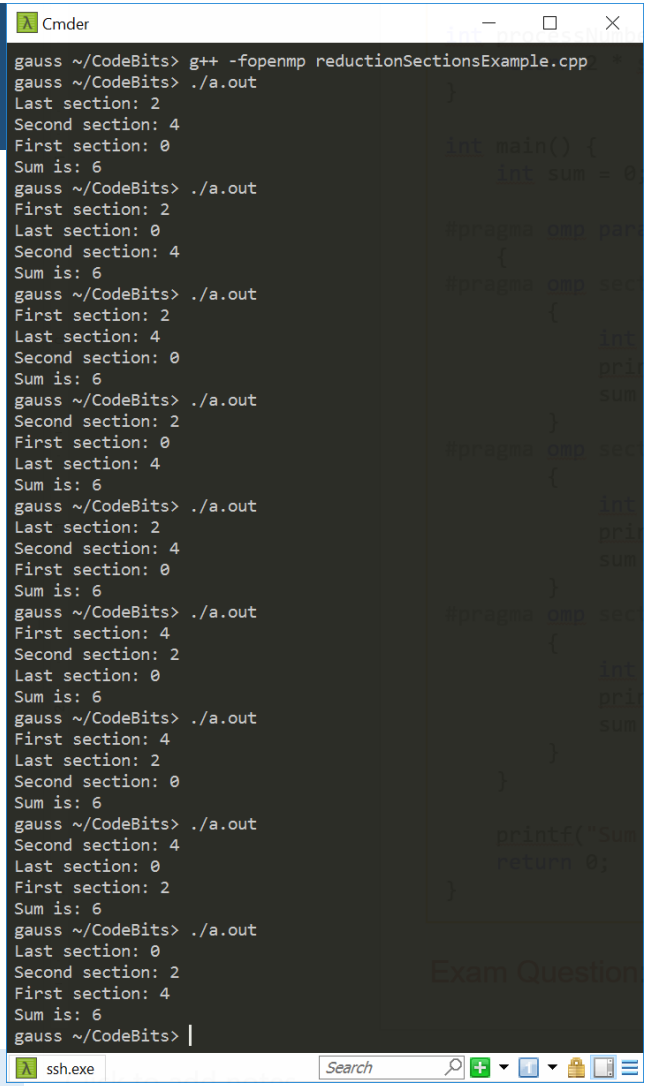
```
$ g++ test.cpp -O3 –fopenmp
$ ./a.out
Pi is: 3.14159
It took this long [s]:
0.000498693
Used this many threads: 4
```

With simd

```
$ lscpu
CPU(s):                          20
On-line CPU(s) list:             0-19
Thread(s) per core:              2
Core(s) per socket:              10
Socket(s):                       1
CPU family:                      6
Model name:                      12th Gen Intel(R) Core(TM) i7-12700H
CPU MHz:                         2688.000
L1d cache:                       480 KiB
L1i cache:                       320 KiB
L2 cache:                        12.5 MiB
L3 cache:                        24 MiB
Flags:                           …, avx2, …
```

OpenMP `reduction` Works Beyond `for` Loops [1/2]

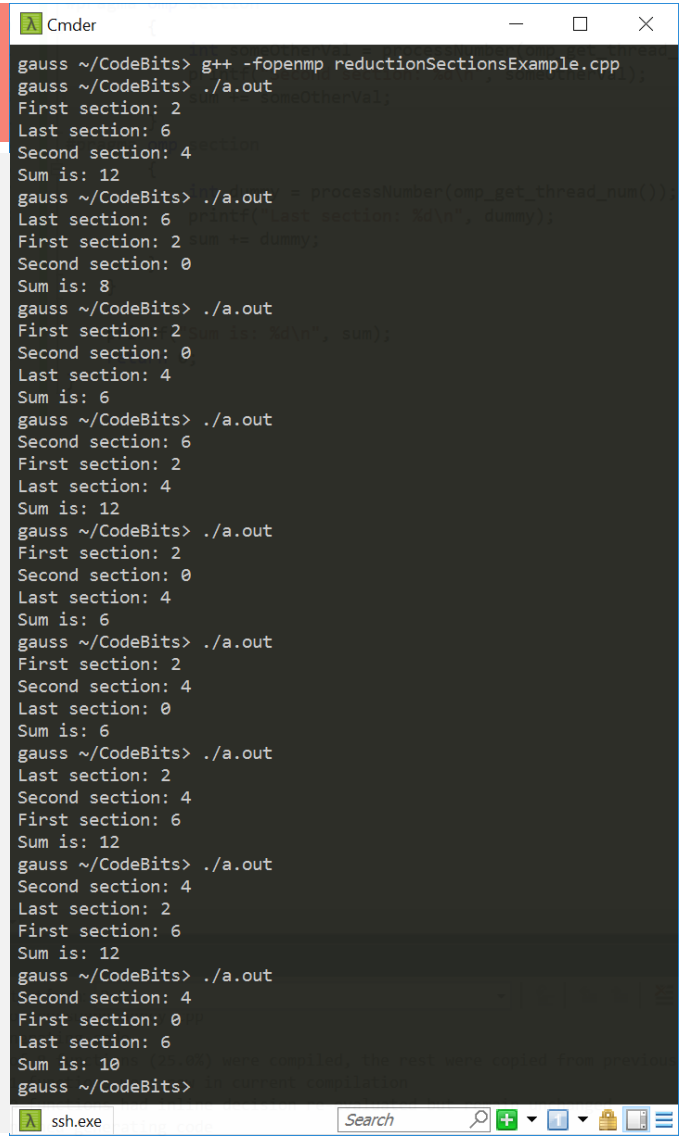
```
#include <cstdio>
#include <omp.h>
int processNumber(int someNumber) {
    return 2 * someNumber;
}
int main() {
    int sum = 0;
    #pragma omp parallel sections num_threads(3) reduction(+:sum)
    {
        #pragma omp section
        {
            int someVal = processNumber(omp_get_thread_num());
            std::printf("First section: %d\n", someVal);
            sum += someVal;
        }
        #pragma omp section
        {
            int someOtherVal = processNumber(omp_get_thread_num());
            std::printf("Second section: %d\n", someOtherVal);
            sum += someOtherVal;
        }
        #pragma omp section
        {
            int dummy = processNumber(omp_get_thread_num());
            std::printf("Last section: %d\n", dummy);
            sum += dummy;
        }
    }
    std::printf("Sum is: %d\n", sum);
    return 0;
}
```



NOTE: `sum` is always the same. What's called "`First section`" is not necessarily executed by thread with smallest id.

Quiz: Why different results for `sum`?

```
#include <cstdio>
#include <omp.h>
int processNumber(int someNumber) {
    return 2 * someNumber;
}
int main() {
    int sum = 0;
    #pragma omp parallel sections num_threads(4) reduction(+:sum)
    {
        #pragma omp section
        {
            int someVal = processNumber(omp_get_thread_num());
            std::printf("First section: %d\n", someVal);
            sum += someVal;
        }
        #pragma omp section
        {
            int someOtherVal = processNumber(omp_get_thread_num());
            std::printf("Second section: %d\n", someOtherVal);
            sum += someOtherVal;
        }
        #pragma omp section
        {
            int dummy = processNumber(omp_get_thread_num());
            std::printf("Last section: %d\n", dummy);
            sum += dummy;
        }
    }
    std::printf("Sum is: %d\n", sum);
    return 0;
}
```



A Histogram Example

[more advanced, uses reduction w/ C++ STL]

```cpp
#include <omp.h>
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <vector>
#include <filesystem>

// Read image file into vector
void read_image(
  const std::filesystem::path& filename,
  std::vector<int>& v
) {

  std::ifstream inf(filename, std::ios::binary);
  for (int i = 0; i < v.size(); i++) {

    inf.read(
      reinterpret_cast<char*>(&v[i]),
      sizeof(int)
    );
  }
}
```

```cpp
int main(int argc, char *argv[]) {
  // Set number of threads
  const auto num_threads = std::atoi(argv[1]);
  omp_set_num_threads(num_threads);

  // Read in Image
  const int d1 = 1800;
  const int d2 = 1200;
  std::vector<int> image(d1 * d2); // initialize vector to dimensions of image
  read_image("picture.img", image);

  // Histogram Counter
  std::vector<int> histCounter = std::vector<int>(7);

  // Declaration of the std::vector addition
#pragma omp declare reduction(vec_int_add : std::vector<int> : \
  std::transform(omp_out.begin(), omp_out.end(), omp_in.begin(), omp_out.begin(),
std::plus<int>())) \
  initializer(omp_priv = omp_orig)


  double startTime = omp_get_wtime(); // Start Clock
#pragma omp parallel for reduction(vec_int_add : histCounter)
  // Perform the histrogram; shared(histCounter)
  for (int component : image)
    (histCounter[component])++;

  double endTime = omp_get_wtime(); // Stop Clock

  // Ouput information
  for (int element : histCounter)
    std::cout << element << std::endl;
  std::cout << num_threads << std::endl;
  std::cout << std::setprecision(16) << (endTime - startTime) * 1000 << std::endl;
  return 0;
}
```